

SecureTicket

Solución B2B de reventa de entradas basada en
contratos inteligentes sobre Stellar/Soroban

Pontificia Universidad Javeriana – Ingeniería de Sistemas

SecureTicket

ESPECIFICACIÓN DE REQUERIMIENTOS DE SOFTWARE

Fecha: 24 de mayo de 2026

Versión del documento: 7.0

Autores:

Juan Diego Arias Durán

Santiago Andres Mesa Niño

Andres Felipe Manosalva Garzón

Juan Camilo Carvajal Rodriguez

Historial de cambios

En esta sección se presenta una tabla que describe la evolución y los cambios realizados al documento desde su primera versión hasta la versión actual.

Versión	Fecha	Sección modificada	Descripción de cambios	Responsable(s)		
1.0	2026-03-11	Sección de Usuarios e Interfaz	Documentación inicial de tipos de usuarios, acceso y pantallas principales.	Andres	Felipe	Mano-salva
1.1	2026-03-11	Requerimientos funcionales (RF 01-20)	Redacción completa de requerimientos funcionales por módulos, con trazabilidad a casos de uso.	Andres	Felipe	Mano-salva
1.2	2026-05-15	Requerimientos de desempeño	Inclusión de requerimientos estáticos y dinámicos, tiempos de respuesta y criterios de medición.	Andres	Felipe	Mano-salva
1.3	2026-05-15	Restricciones de diseño	Documentación de lenguajes, herramientas, arquitectura, hardware/software, base de datos y blockchain según SAD.	Andres	Felipe	Mano-salva
1.4	2026-05-15	Requerimientos no funcionales	Redacción de atributos de calidad (seguridad, integridad, trazabilidad, rendimiento, disponibilidad) con criterios medibles.	Andres	Felipe	Mano-salva
1.5	2026-05-21	Requerimientos de base de datos	Inclusión de tipos de datos, indexación, claves primarias, consultas y frecuencia de acceso.	Andres	Felipe	Mano-salva
1.6	2026-05-21	Proceso de Ingeniería de Requerimientos	Explicación de técnicas, métodos, trazabilidad y gestión de cambios.	Andres	Felipe	Mano-salva
1.7	2026-05-21	Proceso de Verificación	Definición del proceso de verificación y validación de requerimientos, métodos y criterios de medición.	Andres	Felipe	Mano-salva

Cuadro 1: Historial de cambios actualizado

Índice

Historial de cambios	1
1. Introducción	6
1.1. Propósito	6
1.2. Alcance	6
1.3. Definiciones, acrónimos y abreviaciones	6
1.4. Referencias	7
1.5. Apreciación global	8
2. Descripción global	8
2.1. Perspectiva del producto	8
2.1.1. Interfaces con el sistema	9
2.1.2. Interfaces con el usuario	9
2.1.3. Interfaces con el hardware	10
2.1.4. Interfaces con el software	10
2.1.5. Interfaces de comunicación	10
2.1.6. Restricciones de memoria	11
2.1.7. Operaciones	11
2.1.8. Requerimientos de adaptación del sitio	11
2.2. Funciones del producto	12
2.3. Características del usuario	12
2.4. Restricciones	13
2.5. Modelo del dominio	13
2.6. Suposiciones y dependencias	14
2.7. Distribución de requerimientos	14
3. Requerimientos específicos	17
3.1. Características del Producto de Software	17
3.1.1. Funcionalidad 1: Gestión de eventos y contratos	24
3.1.2. Funcionalidad 2: Compra primaria de boletos	24
3.1.3. Funcionalidad 3: Wallets e integración Web3	25
3.1.4. Funcionalidad 4: Blockchain y NFTs	26
3.1.5. Funcionalidad 5: Reventa P2P	27
3.1.6. Funcionalidad 6: Validación QR	28
3.1.7. Funcionalidad 7: Administración y control de boletos	29
3.1.8. Funcionalidad 8: Cruce de funcionalidades	30
3.2. Requerimientos de Desempeño	31
3.2.1. Requerimientos estáticos de desempeño	31
3.2.2. Requerimientos dinámicos de desempeño	32
3.3. Restricciones de Diseño	32
3.3.1. Lenguajes de programación	33
3.3.2. Herramientas de análisis, diseño e implementación	33
3.3.3. Restricciones de Hardware y Software	33

3.3.4.	Restricciones de arquitectura	34
3.3.5.	Restricciones de Blockchain y contratos inteligentes	34
3.3.6.	Restricciones de la base de datos	34
3.3.7.	Restricciones de autenticacion y seguridad	35
3.3.8.	Restricciones de despliegue	35
3.4.	Atributos del Sistema de Software (No funcionales)	35
3.4.1.	Seguridad	35
3.4.2.	Integridad	36
3.4.3.	Trazabilidad	36
3.4.4.	Modificabilidad	37
3.4.5.	Disponibilidad	37
3.5.	Requerimientos de la base de datos	38
3.5.1.	Tipos de datos almacenados	38
3.5.2.	Tipo de consultas utilizadas	38
3.5.3.	Indexación de los datos	39
3.5.4.	Uso de las Primary Key	39
3.5.5.	Frecuencia de acceso y conexiones	40
4.	Proceso de Ingeniería de Requerimientos	40
4.1.	Técnicas utilizadas	40
4.2.	Métodos de gestión de cambios	41
5.	Proceso de Verificación	41
5.1.	Verificación de requerimientos funcionales	42
5.2.	Verificación de requerimientos no funcionales y de desempeño	42
6.	Referencias	43

Índice de cuadros

1.	Historial de cambios actualizado	1
2.	Acrónimos y definiciones relevantes	7
3.	Interfaces de software previstas	10
4.	Siglas de los módulos del sistema	15
5.	Mapecto de requerimientos funcionales con los módulos del sistema	15
6.	Mapecto de requerimientos no funcionales con los módulos del sistema	16
7.	Documentación de Requerimientos RF-01	17
8.	Documentación de Requerimientos RF-02	17
9.	Documentación de Requerimientos RF-03	18
10.	Documentación de Requerimientos RF-04	18
11.	Documentación de Requerimientos RF-05	18
12.	Documentación de Requerimientos RF-06	19
13.	Documentación de Requerimientos RF-07	19
14.	Documentación de Requerimientos RF-08	19
15.	Documentación de Requerimientos RF-09	20
16.	Documentación de Requerimientos RF-10	20
17.	Documentación de Requerimientos RF-11	20
18.	Documentación de Requerimientos RF-12	21
19.	Documentación de Requerimientos RF-13	21
20.	Documentación de Requerimientos RF-14	21
21.	Documentación de Requerimientos RF-15	22
22.	Documentación de Requerimientos RF-16	22
23.	Documentación de Requerimientos RF-17	22
24.	Documentación de Requerimientos RF-18	23
25.	Documentación de Requerimientos RF-19	23
26.	Documentación de Requerimientos RF-20	23
27.	Formato de documentación del Modelo del Dominio	24
28.	Formato de documentación del Modelo del Dominio	25
29.	Formato de documentación del Modelo del Dominio	26
30.	Formato de documentación del Modelo del Dominio	27
31.	Formato de documentación del Modelo del Dominio	28
32.	Formato de documentación del Modelo del Dominio	29
33.	Formato de documentación del Modelo del Dominio	30
34.	Formato de documentación del Modelo del Dominio	31
35.	Requerimientos de desempeño	32
36.	Requerimientos de desempeño dinámicos	32
37.	Documentación de Requerimientos No Funcionales RNF-01	36
38.	Documentación de Requerimientos No Funcionales RNF-02	36
39.	Documentación de Requerimientos No Funcionales RNF-03	37
40.	Documentación de Requerimientos No Funcionales RNF-04	37
41.	Documentación de Requerimientos No Funcionales RNF-05	38
42.	Modelo de datos: entidades y campos principales	38
43.	Métodos de verificación y validación	43

Índice de figuras

1. Introducción

1.1. Propósito

El presente documento corresponde a la *Especificación de Requerimientos de Software* (SRS) del sistema **Tickets-NFT B2B**. Su propósito es:

- Describir de manera completa, consistente y verificable los requerimientos del sistema.
- Servir como acuerdo formal entre los interesados (equipo de tesis, docentes, potenciales aliados de boletería y desarrolladores) sobre qué debe hacer el sistema y bajo qué restricciones.
- Proveer la base para las actividades posteriores de diseño, implementación, pruebas, documentación y mantenimiento.

Este SRS describe únicamente la primera versión del prototipo académico del sistema, ejecutado sobre la *testnet* de Stellar.

1.2. Alcance

El sistema **Tickets-NFT B2B** es un módulo de software que se integra con plataformas de boletería existentes para soportar el mercado secundario de entradas. La versión descrita en este documento tiene como alcance:

- Emitir entradas a eventos como *tokens no fungibles* (NFTs) únicos sobre la red Stellar (*testnet*), utilizando contratos inteligentes Soroban.
- Gestionar la transferencia de propiedad de dichos NFTs entre usuarios en el contexto de reventa B2B/B2C.
- Aplicar reglas de negocio configurables (comisiones por reventa, límites de precio, ventanas de reventa, políticas anti-especulación).
- Exponer una API REST para que la plataforma de boletería integre estas capacidades dentro de sus procesos de venta primaria y secundaria.
- Proveer una interfaz web de demostración para administradores de plataforma y compradores/vendedores.

Quedan fuera del alcance de esta versión: despliegue en *mainnet*, integraciones reales con pasarelas de pago fiat, aplicaciones móviles nativas, procesos de identificación y verificación de usuarios (KYC/AML) y aspectos comerciales o legales más allá del contexto académico.

1.3. Definiciones, acrónimos y abreviaciones

En la Tabla 2 se definen los términos y acrónimos más relevantes utilizados en este SRS. Cualquier término adicional deberá incorporarse a esta tabla en versiones posteriores del documento.

Término	Definición
NFT	<i>Non-Fungible Token</i> . Activo digital único emitido sobre una red blockchain. En el sistema, cada ticket se representa como un NFT.
Stellar	Red blockchain orientada a pagos y emisión de activos digitales, utilizada como infraestructura base del sistema.
Soroban	Plataforma de contratos inteligentes de Stellar utilizada para implementar la lógica de emisión y reventa de tickets.
Tickets-NFT B2B	Nombre corto del sistema de reventa B2B de entradas con NFTs en Stellar.
SRS	<i>Software Requirements Specification</i> . Este documento.
SPMP	<i>Software Project Management Plan</i> del proyecto Tickets-NFT B2B.
API	<i>Application Programming Interface</i> . Conjunto de endpoints expuestos por el backend.
B2B	<i>Business to Business</i> . Modelo en el que el sistema se ofrece a otras empresas (plataformas de boletería).
B2C	<i>Business to Consumer</i> . Interacción entre la plataforma y el consumidor final.
RF	Requerimiento funcional.
RNF	Requerimiento no funcional.
Testnet	Red de pruebas de Stellar, sin valor monetario real.

Cuadro 2: Acrónimos y definiciones relevantes

1.4. Referencias

Las referencias empleadas para la elaboración de este SRS se encuentran organizadas en el sistema de gestión bibliográfica del Overleaf. Entre las más relevantes se encuentran:

- Plantilla *SRSINGESOFT_V2.0* de IronWorks – Ingeniería de Sistemas, Pontificia Universidad Javeriana.
- Plan de Gestión del Proyecto de Software (SPMP) del sistema Tickets-NFT B2B.
- Documento “Mercado de reventa, Stellar y Blockchain”, que describe el contexto del mercado de reventa de entradas y motiva el uso de Stellar/Soroban.
- Documentación oficial de Stellar y Soroban (protocolos, SDK, guías de desarrollo y mejores prácticas).
- Normativa colombiana relacionada con comercio electrónico y protección al consumidor (Ley 1480 de 2011 y reglamentaciones asociadas).

1.5. Apreciación global

El resto del documento se estructura de la siguiente forma:

- La **Sección 2** presenta una descripción global del sistema, incluyendo su perspectiva dentro del entorno de boletería, las funciones de alto nivel, las características de los usuarios, restricciones, modelo del dominio, suposiciones y dependencias, y una distribución preliminar de requerimientos.
- La **Sección 3** detalla los requerimientos específicos del producto, incluyendo interfaces externas, requerimientos funcionales, requerimientos de desempeño, restricciones de diseño, atributos de calidad y requerimientos de base de datos.
- La **Sección 4** describe el proceso de ingeniería de requerimientos utilizado para obtener, analizar y documentar los requerimientos del sistema.
- La **Sección 5** expone el proceso de verificación y validación de los requerimientos.
- La **Sección 6** incluye anexos con material complementario (diagramas de modelo de dominio, plantillas de casos de uso, matrices de trazabilidad, etc.).

2. Descripción global

Esta sección describe el contexto general del producto, los usuarios objetivo y las restricciones que condicionan los requerimientos. No pretende ser una lista exhaustiva de requerimientos, sino el marco conceptual sobre el cual se apoya la Sección 3.

2.1. Perspectiva del producto

El sistema **Tickets-NFT B2B** es un *producto nuevo* que se integra como componente de backend dentro del ecosistema de las plataformas de boletería actuales. No reemplaza completamente los sistemas existentes, sino que provee servicios especializados para:

- Representar tickets como NFTs en Stellar.
- Gestionar el mercado secundario de reventa de esos tickets.
- Exponer la trazabilidad de propiedad de cada ticket.

Desde el punto de vista de la plataforma de boletería, el sistema se percibe como un servicio externo accesible mediante una API REST, responsable de la interacción con la red Stellar/Soroban y del manejo de la lógica de reventa.

2.1.1. Interfaces con el sistema

Los principales sistemas externos con los que interactúa Tickets-NFT B2B son:

- **Plataforma de boletería aliada:** sistema que gestiona la venta primaria de entradas (creación de eventos, asignación de localidades, precios base). Interactúa con el backend de Tickets-NFT B2B a través de una API REST para:
 - Registrar eventos y lotes de tickets que serán tokenizados.
 - Consultar el estado de los tickets y sus historiales de reventa.
- **Red Stellar (testnet) y nodos Soroban:** infraestructura de libro mayor distribuido donde se emiten y transfieren los NFTs. El backend utiliza los SDK y endpoints oficiales para enviar transacciones y consultar estado.

El motor de base de datos utilizado por Tickets-NFT B2B se considera parte del mismo producto y no se trata como un sistema externo.

2.1.2. Interfaces con el usuario

La interacción de los usuarios se realiza principalmente mediante una **aplicación web** accesible desde navegadores modernos. Se distinguen dos tipos de interfaz:

- **Interfaz de administrador de plataforma:**
 - Permite crear y administrar eventos y lotes de tickets.
 - Configura políticas de reventa (porcentaje de comisión, límites de precio, periodo en el que la reventa está permitida).
 - Consulta reportes resumidos de actividad de reventa.
- **Interfaz de comprador/vendedor:**
 - Permite visualizar los tickets que posee el usuario.
 - Permite publicar tickets en venta en el mercado secundario.
 - Permite adquirir tickets en reventa y verificar su autenticidad y trazabilidad.

Requisitos preliminares de interfaz:

- Se prioriza compatibilidad con navegadores Google Chrome y Mozilla Firefox en sus versiones actuales.
- Resolución mínima recomendada: 1366x768 píxeles.
- La interfaz debe presentar mensajes de error claros y evitar, en lo posible, exponer terminología técnica de blockchain al usuario final.

2.1.3. Interfaces con el hardware

El sistema está pensado como una aplicación web cliente-servidor que no requiere hardware especializado. Las interfaces relevantes son:

- **Equipos cliente:** computadores de escritorio o portátiles, y potencialmente dispositivos móviles, con tarjeta de red Ethernet o WiFi que les permita conectarse a Internet.
- **Servidor de aplicación:** máquina virtual o servidor en la nube capaz de ejecutar el backend, la base de datos y las herramientas de despliegue del contrato Soroban.

A nivel preliminar se asume que el servidor cuenta con al menos 2 vCPU, 4 GB de RAM y 20 GB de almacenamiento disponible para el prototipo.

2.1.4. Interfaces con el software

El sistema depende de varios productos de software. La Tabla 3 resume los principales componentes previstos en esta etapa (sujetos a cambios en el documento de diseño).

Producto	Versión	Fuente	Propósito
SO servidor (Ubuntu)	22.04 LTS	Sitio oficial Ubuntu	Sistema operativo del servidor de aplicación y base de datos.
PostgreSQL	15.x	Sitio oficial PostgreSQL	Almacenamiento de información off-chain (usuarios, eventos, configuración, vínculos a NFTs).
Entorno backend (p.ej. Node.js)	20.x	Sitio oficial Node.js	Ejecución de la API REST y lógica de negocio fuera de la blockchain.
SDK Stellar/Soroban	Última estable	Documentación oficial Stellar	Firma y envío de transacciones a la red Stellar y al contrato Soroban.
Navegadores web	Versiones actuales	Sitios oficiales	Ejecución del frontend en los equipos cliente.

Cuadro 3: Interfaces de software previstas

2.1.5. Interfaces de comunicación

La comunicación entre los componentes del sistema se realiza mediante:

- **HTTP/HTTPS** sobre protocolo **TCP/IP** para:
 - Comunicación entre el frontend web y el backend.
 - Consumo de la API REST por parte de la plataforma de boletería.

- Acceso del backend a endpoints públicos de la red Stellar y servicios asociados.
- **Conexiones de base de datos** sobre **TCP/IP** entre el backend y el servidor PostgreSQL.

Los puertos específicos y configuraciones de firewall se detallarán en el documento de diseño e instalación.

2.1.6. Restricciones de memoria

Aunque no se prevé un consumo intensivo de memoria en el contexto académico, se establecen los siguientes mínimos recomendados:

- Servidor de aplicación: al menos 4 GB de memoria RAM disponible para backend, base de datos y servicios auxiliares.
- Equipos cliente: al menos 4 GB de RAM para ejecutar un navegador moderno sin afectar la experiencia de usuario.

En futuras versiones, si el número de usuarios o eventos crece significativamente, será necesario ajustar estas cifras.

2.1.7. Operaciones

Se identifican los siguientes modos de operación relevantes:

- **Modo administrador:** orientado a la gestión de eventos, lotes de tickets y políticas de reventa.
- **Modo comprador/vendedor:** orientado a la gestión del inventario de tickets y las operaciones de compra/venta.

Adicionalmente, el sistema debe soportar operaciones periódicas como:

- Generación de respaldos de la base de datos (por ejemplo, diarios).
- Registro de logs de actividad para fines de auditoría y depuración.

2.1.8. Requerimientos de adaptación del sitio

Para adaptarse a distintas plataformas de boletería, el sistema deberá permitir configurar, sin modificar el código fuente:

- Parámetros de identificación de la plataforma (nombre comercial, logotipo, URL de retorno).
- Moneda de referencia y formato de precios.
- Reglas por defecto de reventa (comisión base, límite máximo de precio, ventana de reventa).

Estos parámetros se gestionarán mediante archivos de configuración o tablas de base de datos, documentadas en la guía de instalación y operación.

2.2. Funciones del producto

A un nivel alto, el sistema Tickets-NFT B2B proporciona las siguientes funciones:

- **Gestión de eventos y lotes de tickets:** creación, consulta y actualización de eventos y de los lotes de tickets asociados.
- **Emisión de tickets NFT:** generación de NFTs en la red Stellar y registro de su relación con los tickets del sistema.
- **Gestión de inventario de tickets:** asignación de tickets a usuarios, visualización de tickets disponibles y vendidos.
- **Gestión de mercado secundario:** publicación de tickets en venta, consulta de ofertas y ejecución de operaciones de compra/venta.
- **Cálculo y distribución de comisiones:** aplicación de reglas configurables al realizar reventas.
- **Consulta de trazabilidad:** visualización del historial de propiedad y transacciones asociadas a un ticket NFT.

Estas funciones se refinarán en la Sección 3 como requerimientos funcionales individuales.

2.3. Características del usuario

Se identifican las siguientes clases de usuarios:

- **Administrador de plataforma:**
 - Perfil: personal de la plataforma de boletería o del equipo de operación del sistema.
 - Responsabilidades: configurar eventos, lotes de tickets y políticas de reventa; monitorear la actividad general del sistema.
 - Nivel técnico: familiarizado con herramientas web de administración; no se asume conocimiento profundo de blockchain.
- **Comprador/Vendedor:**
 - Perfil: usuario final que compra entradas y, en caso necesario, revende sus tickets.
 - Responsabilidades: gestionar su inventario de tickets, publicar ofertas de venta y comprar tickets en el mercado secundario.
 - Nivel técnico: conocimientos básicos de navegación web; se espera que el sistema abstraiga los detalles técnicos de la red Stellar.

En futuras versiones del SRS se podrá incluir una tabla para cada tipo de usuario, especificando capacidades, necesidades de capacitación y nivel de seguridad requerido.

2.4. Restricciones

Las restricciones que condicionan el diseño y los requerimientos del sistema se clasifican de la siguiente forma:

Restricciones generales

- El sistema se desarrolla como parte de un proyecto académico con tiempo y recursos limitados, lo cual acota el alcance funcional y el nivel de robustez esperado.
- La versión descrita está orientada a funcionar exclusivamente sobre la *testnet* de Stellar.

Restricciones de software

- Uso de tecnologías libremente disponibles o con licencias adecuadas para fines académicos.
- El sistema debe ejecutarse en entornos compatibles con las versiones especificadas de SO, DBMS y frameworks listados en la Tabla 3.

Restricciones de hardware

- Se asume disponibilidad de un servidor con recursos mínimos indicados en la sección de interfaces con el hardware.
- Los clientes deben disponer de equipos con capacidad para ejecutar navegadores modernos sin degradación severa del rendimiento.

2.5. Modelo del dominio

El modelo de dominio describe los conceptos principales del problema y sus relaciones, independientemente de la implementación final. De manera preliminar se identifican las siguientes entidades:

- **Evento:** representación de un concierto, espectáculo u otro tipo de evento.
- **Ticket:** derecho de asistir a un evento en una localidad específica; se vincula a un NFT.
- **Usuario:** persona o entidad que compra, posee o revende tickets.
- **Transacción de reventa:** operación mediante la cual un ticket cambia de propietario en el mercado secundario.
- **Política de reventa:** conjunto de reglas asociadas a un evento o plataforma, que definen comisiones y restricciones de reventa.

En una versión posterior de este documento se añadirá un diagrama de modelo de dominio y una tabla por cada entidad, con sus atributos, tipo de dato y descripción, siguiendo el formato propuesto por la plantilla.

2.6. Suposiciones y dependencias

Suposiciones

- Se asume que la plataforma de boletería interesada cuenta ya con un sistema de venta primaria de tickets y un modelo de datos básico para eventos y usuarios.
- Se asume que los usuarios finales disponen de conexión a Internet estable para usar la interfaz web.
- Se asume que la red Stellar testnet estará disponible y con un nivel razonable de rendimiento durante el periodo de desarrollo y pruebas.

Dependencias

- El correcto funcionamiento del sistema depende de la disponibilidad de la red Stellar testnet y de los servicios asociados a Soroban.
- La API REST que expone Tickets-NFT B2B depende de la infraestructura de red y de los servidores donde se despliegue el backend.
- Cualquier cambio significativo en los SDK o en los modelos de transacción de Stellar/Soroban puede requerir ajustes en el diseño y en los requerimientos.

2.7. Distribución de requerimientos

De forma preliminar, los requerimientos del sistema se distribuyen entre los siguientes módulos funcionales:

- **Módulo de gestión de eventos y tickets:** agrupa los requerimientos relacionados con la creación y administración de eventos y lotes de tickets.
- **Módulo de emisión NFT:** agrupa los requerimientos que definen cómo se crean y vinculan los NFTs en Stellar para cada ticket.
- **Módulo de mercado secundario:** agrupa los requerimientos sobre publicación de ofertas, compra/venta y trazabilidad.
- **Módulo de configuración y políticas:** agrupa los requerimientos sobre comisiones, restricciones de reventa y parámetros de integración con plataformas externas.
- **Módulo de usuarios y autenticación:** agrupa los requerimientos de manejo de cuentas de usuario y control de acceso.

Cuadro 4: Siglas de los módulos del sistema

Sigla	Módulo
M1	Gestión de eventos y tickets
M2	Emisión NFT
M3	Mercado secundario
M4	Configuración y políticas
M5	Usuarios y autenticación

Cuadro 5: Mapeo de requerimientos funcionales con los módulos del sistema

ID	Requerimiento resumido	M1	M2	M3	M4	M5	Observación
RF-01	Crear evento y desplegar contrato Soroban	X	X		X		Registra el evento y su contrato de gestión de boletos.
RF-02	Comprar boleto en venta primaria	X				X	Cubre catálogo, carrito, checkout y asignación del boleto al usuario.
RF-03	Listar boleto propio para reventa		X	X	X	X	Valida propiedad, estado vigente y publicación en el mercado P2P.
RF-04	Comprar boleto en reventa atómica		X	X	X	X	Ejecuta la transferencia segura con firma Freighter y actualización de propiedad.
RF-05	Distribuir comisiones de reventa		X	X	X		Aplica reglas del contrato para pagos a vendedor, organizador y plataforma.
RF-06	Ejecutar burn/remint de versión		X	X			Reemplaza la versión anterior del boleto tras una reventa exitosa.
RF-07	Verificar y redimir boleto en puerta	X	X			X	Valida el QR vigente y marca el boleto como usado.
RF-08	Invalidar boleto	X	X		X		Bloquea boletos comprometidos para reventa, transferencia o ingreso.
RF-09	Conectar wallet Freighter		X			X	Vincula la cuenta Web2 con una dirección Stellar.
RF-10	Asegurar boleto en blockchain		X	X		X	Emite o registra la representación NFT antes de operaciones Web3.
RF-11	Publicar boleto propio en reventa			X	X	X	Expone el boleto en el mercado secundario con precio definido.
RF-12	Cancelar publicación de reventa			X	X	X	Permite retirar del mercado un boleto listado por su propietario.
RF-13	Consultar boletos en reventa			X			Lista los boletos disponibles para compra P2P.
RF-14	Comprar boleto en reventa mediante Freighter		X	X	X	X	Firma y ejecuta la compra P2P desde la wallet del comprador.
RF-15	Invalidar versión anterior y generar nueva versión		X	X			Mantiene un único QR vigente después de la reventa.
RF-16	Distribuir comisiones programadas		X	X	X		Registra montos, comisiones y wallets involucradas.
RF-17	Consultar historial de ventas P2P	X	X	X		X	Permite al vendedor auditar sus operaciones de reventa.
RF-18	Generar QR único para boleto válido	X	X				Asocia el QR con el identificador y versión vigente del boleto.
RF-19	Escanear QR en puerta	X				X	Permite al verificador leer el QR desde la interfaz de escáner.
RF-20	Rechazar QR usados, invalidados o vencidos	X	X		X	X	Evita reutilización, fraude y uso de versiones antiguas por reventa.

Cuadro 6: Mapeo de requerimientos no funcionales con los módulos del sistema

ID	Requerimiento resumido	M1	M2	M3	M4	M5	Observación
RNF-01	Procesar la compra de boletos en menos de 5 segundos	X		X			Aplica a compras y reventas.
RNF-02	Soportar al menos 100 usuarios concurrentes	X		X		X	Involucra uso simultáneo del sistema.
RNF-03	Tener disponibilidad mínima del 99.5 % mensual	X	X	X	X	X	Afecta a todo el sistema.
RNF-04	Bloquear transacciones tras 3 intentos fallidos consecutivos		X	X	X	X	Aplica a operaciones sensibles y validación de usuarios.
RNF-05	Cifrar comunicaciones mediante TLS 1.2 o superior	X	X	X	X	X	Protege la comunicación en todo el sistema.
RNF-06	Soportar al menos 2400 transacciones diarias	X		X			Aplica a compras, ventas y reventas.
RNF-07	Completar el proceso de compra en máximo 5 pasos	X		X		X	Relacionado con facilidad de uso.
RNF-08	Permitir aprendizaje del usuario en menos de 15 minutos	X		X		X	Aplica a la experiencia general del usuario.
RNF-09	Mantener una tasa de fallos menor al 0.5 %	X	X	X			Aplica principalmente a las transacciones.
RNF-10	Garantizar atomicidad en el 100 % de las transacciones	X	X	X	X		Evita errores en compras, reventas y transferencias.
RNF-11	Tener cobertura de pruebas mínima del 70 %	X	X	X	X	X	Aplica al mantenimiento del sistema completo.
RNF-12	Ser compatible con navegadores web	X		X		X	Permite acceso desde distintos navegadores.
RNF-13	Integrarse con contratos Soroban mediante API REST		X	X	X		Relacionado con blockchain e integración externa.
RNF-14	Almacenar logs por mínimo 12 meses	X	X	X	X	X	Permite auditoría y seguimiento del sistema.
RNF-15	Cerrar sesión tras 30 minutos de inactividad					X	Pertenece directamente al control de sesiones.
RNF-16	No superar 5 horas de inactividad mensual	X	X	X	X	X	Relacionado con disponibilidad general.
RNF-17	Verificar boletos en menos de 5 segundos	X	X				Aplica al proceso de validación de boletos.
RNF-18	Soportar varios eventos simultáneos	X		X	X		Involucra eventos, ventas y reglas de operación.
RNF-19	Recuperarse de fallos en menos de 20 minutos	X	X	X	X	X	Aplica a recuperación del sistema completo.
RNF-20	Conservar historial de boletos por mínimo 5 años	X	X	X		X	Relacionado con trazabilidad de boletos y usuarios.

3. Requerimientos específicos

En esta sección se presenta la especificación detallada de los requerimientos del sistema, tomando como base la distribución definida en la sección 2.7. El objetivo es describir de manera clara, completa y verificable las funcionalidades y restricciones que debe cumplir el sistema.

Los requerimientos se organizan de acuerdo con los módulos principales del sistema: gestión de eventos y tickets, emisión NFT, mercado secundario, configuración y políticas, y usuarios y autenticación. Esta organización facilita la trazabilidad, validación, mantenimiento y futura actualización de cada requerimiento.

La especificación funcional vigente usa veinte requerimientos funcionales (RF-01 a RF-20) y seis casos de uso significativos (CU-01 a CU-06), alineados con el SDD. Las tablas siguientes constituyen la versión canónica de la especificación detallada y evitan identificadores funcionales adicionales fuera de la matriz de trazabilidad.

3.1. Características del Producto de Software

# Requerimiento	RF-01	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-01 Crear evento y desplegar contrato		
Actor Principal	Admin Plataforma		
Descripción	El sistema debe permitir que el administrador cree un evento y despliegue el contrato correspondiente en Soroban para la gestión de boletos.		
Entradas	Título del evento, fecha, venue, capacidad, imagen, categoría, ciudad y datos requeridos para el contrato.		
Proceso	El sistema registra el evento en la base de datos y despliega los contratos necesarios en Stellar Testnet mediante Soroban.		
Salidas	Evento creado, contrato desplegado y dirección del contrato almacenada en el sistema.		
Razón	Es necesario que cada evento tenga una configuración propia para administrar boletos, compras, reventas e invalidaciones.		
Criterio de medición	El evento aparece en el catálogo y el contrato queda registrado correctamente en la base de datos.		
Prioridad	Alta	Módulo Asociado	Gestión de eventos y contratos
Versión	2.0	Fecha	[19/05/2026]

Cuadro 7: Documentación de Requerimientos RF-01

# Requerimiento	RF-02	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-02 Compra primaria de boleto		
Actor Principal	Usuario		
Descripción	El sistema debe permitir que el usuario compre boletos disponibles directamente desde la plataforma.		
Entradas	Evento seleccionado, tipo de boleto, cantidad, información del carrito y datos del checkout.		
Proceso	El sistema valida la disponibilidad, procesa el checkout con pago simulado, genera la orden y asigna el boleto al usuario.		
Salidas	Orden de compra creada, boleto asignado al usuario y entrada disponible en la sección Mis Entradas.		
Razón	Esta es la funcionalidad principal para que los usuarios puedan adquirir entradas de forma inicial.		
Criterio de medición	Al finalizar la compra, el usuario puede visualizar el boleto adquirido en su cuenta.		
Prioridad	Alta	Módulo Asociado	Compra primaria de boletos
Versión	2.0	Fecha	[19/05/2026]

Cuadro 8: Documentación de Requerimientos RF-02

# Requerimiento	RF-03	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-03 Listar boleto para reventa		
Actor Principal	Usuario		
Descripción	El sistema debe permitir que un usuario publique un boleto propio en el mercado de reventa P2P.		
Entradas	Boleto seleccionado, precio de reventa y wallet del usuario.		
Proceso	El sistema verifica que el boleto pertenezca al usuario, que no esté usado ni invalidado, y lo marca como disponible para reventa.		
Salidas	Boleto publicado en el mercado P2P con precio visible para otros usuarios.		
Razón	Permite que los usuarios puedan revender boletos que ya no utilizarán, conservando la trazabilidad de propiedad.		
Criterio de medición	El boleto aparece en la sección de Reventa P2P y su estado cambia a “en venta”.		
Prioridad	Alta	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 9: Documentación de Requerimientos RF-03

# Requerimiento	RF-04	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica		
Actor Principal	Usuario		
Descripción	El sistema debe permitir que un usuario compre un boleto publicado por otro usuario en el mercado de reventa P2P.		
Entradas	Boleto en reventa, wallet del comprador y firma de transacción mediante Freighter.		
Proceso	El sistema ejecuta la compra en blockchain, actualiza la propiedad del boleto, registra la transacción y sincroniza el nuevo estado en la base de datos.		
Salidas	Boleto transferido al nuevo comprador, versión anterior reemplazada y operación registrada.		
Razón	Garantiza que la reventa se realice de forma segura, trazable y sin duplicar la propiedad del boleto.		
Criterio de medición	Después de la compra, el boleto aparece en Mis Entradas del comprador y deja de estar disponible en el mercado P2P.		
Prioridad	Alta	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 10: Documentación de Requerimientos RF-04

# Requerimiento	RF-05	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica — incluye Distribuir comisiones		
Actor Principal	Sistema / Contrato Soroban		
Descripción	El sistema debe distribuir las comisiones generadas durante una compra en reventa P2P según las reglas definidas en el contrato.		
Entradas	Precio de reventa, datos del comprador, vendedor y parámetros de comisión.		
Proceso	El contrato calcula y distribuye los montos correspondientes entre las partes involucradas.		
Salidas	Comisión distribuida y transacción registrada en blockchain y base de datos.		
Razón	Permite controlar los ingresos derivados de la reventa y mantener reglas transparentes dentro del mercado secundario.		
Criterio de medición	La operación de reventa registra correctamente el monto, la comisión y las wallets involucradas.		
Prioridad	Alta	Módulo Asociado	Reventa P2P / Contrato Soroban
Versión	2.0	Fecha	[19/05/2026]

Cuadro 11: Documentación de Requerimientos RF-05

# Requerimiento	RF-06	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica — incluye Burn/Remint versión		
Actor Principal	Sistema / Contrato Soroban		
Descripción	El sistema debe invalidar la versión anterior del boleto y generar una nueva versión cuando se complete una compra en reventa.		
Entradas	Identificador del boleto, versión actual, wallet del vendedor y wallet del comprador.		
Proceso	El contrato quema o invalida la versión anterior del boleto y genera una nueva versión asociada al nuevo propietario.		
Salidas	Nueva versión del boleto, nuevo propietario registrado y QR anterior invalidado.		
Razón	Evita el uso de boletos antiguos después de una reventa y garantiza que solo el comprador final tenga un boleto válido.		
Criterio de medición	El QR antiguo es rechazado y el nuevo boleto queda asociado a la wallet del comprador.		
Prioridad	Alta	Módulo Asociado	Reventa P2P / Blockchain y NFTs
Versión	2.0	Fecha	[19/05/2026]

Cuadro 12: Documentación de Requerimientos RF-06

# Requerimiento	RF-07	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-05 Verificar y redimir boleto en puerta		
Actor Principal	Verificador / Personal Evento		
Descripción	El sistema debe permitir que el personal del evento escanee el código QR de un boleto para verificar su validez y redimirlo en la entrada.		
Entradas	Código QR del boleto.		
Proceso	El sistema consulta la información del boleto, valida que pertenezca a la versión vigente y verifica que no esté usado, invalidado o vencido por reventa.		
Salidas	Resultado de validación: boleto aceptado o rechazado. Si es aceptado, queda marcado como usado.		
Razón	Permite controlar el ingreso al evento y evitar el uso duplicado o fraudulento de entradas.		
Criterio de medición	Un boleto válido permite el ingreso una sola vez; un boleto usado, invalidado o con QR antiguo es rechazado.		
Prioridad	Alta	Módulo Asociado	Validación de boletos
Versión	2.0	Fecha	[19/05/2026]

Cuadro 13: Documentación de Requerimientos RF-07

# Requerimiento	RF-08	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-06 Invalidar boleto		
Actor Principal	Organizador / Ticketera		
Descripción	El sistema debe permitir que el organizador invalide un boleto cuando sea necesario, por ejemplo, por fraude, error, cancelación o uso indebido.		
Entradas	Identificador del boleto y motivo de invalidación.		
Proceso	El sistema cambia el estado del boleto a invalidado y bloquea su uso en procesos de reventa, transferencia o validación en puerta.		
Salidas	Boleto invalidado y registrado como no válido dentro del sistema.		
Razón	Es necesario que la ticketera pueda controlar situaciones excepcionales y proteger la autenticidad del evento.		
Criterio de medición	Un boleto invalidado no puede ser escaneado, revendido ni utilizado para ingresar al evento.		
Prioridad	Alta	Módulo Asociado	Administración y control de boletos
Versión	2.0	Fecha	[19/05/2026]

Cuadro 14: Documentación de Requerimientos RF-08

# Requerimiento	RF-09	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-03 Listar boleto para reventa		
Actor Principal	Usuario		
Descripción	El sistema debe permitir que el usuario conecte su wallet Freighter a su cuenta.		
Entradas	Dirección pública de la wallet y autorización desde Freighter.		
Proceso	El sistema vincula la cuenta Web2 del usuario con su dirección pública Stellar.		
Salidas	Cuenta vinculada a una wallet.		
Razón	La wallet es necesaria para realizar operaciones Web3 como asegurar, revender o comprar boletos en reventa.		
Criterio de medición	La wallet queda registrada y asociada únicamente al usuario autenticado.		
Prioridad	Alta	Módulo Asociado	Wallets
Versión	2.0	Fecha	[19/05/2026]

Cuadro 15: Documentación de Requerimientos RF-09

# Requerimiento	RF-10	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-03 Listar boleto para reventa		
Actor Principal	Usuario		
Descripción	El sistema debe permitir asegurar un boleto en blockchain antes de ser revendido.		
Entradas	Boleto seleccionado y wallet del usuario.		
Proceso	El sistema registra el boleto en blockchain mediante contratos Soroban y emite su representación NFT.		
Salidas	Boleto asegurado en blockchain y asociado a la wallet del usuario.		
Razón	Es necesario garantizar autenticidad y trazabilidad antes de permitir la reventa.		
Criterio de medición	El boleto queda asociado a un contrato, token NFT y propietario.		
Prioridad	Alta	Módulo Asociado	Blockchain y NFTs
Versión	2.0	Fecha	[19/05/2026]

Cuadro 16: Documentación de Requerimientos RF-10

# Requerimiento	RF-11	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-03 Listar boleto para reventa		
Actor Principal	Usuario		
Descripción	El sistema debe permitir publicar un boleto propio en el mercado de reventa P2P.		
Entradas	Boleto seleccionado, precio de reventa y wallet del propietario.		
Proceso	El sistema verifica la propiedad del boleto, valida que no esté usado ni invalidado y lo marca como disponible para reventa.		
Salidas	Boleto listado en el mercado P2P.		
Razón	Permite que un usuario revenda un boleto que ya no utilizará.		
Criterio de medición	El boleto aparece en el módulo de Reventa P2P con estado “en venta”.		
Prioridad	Alta	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 17: Documentación de Requerimientos RF-11

# Requerimiento	RF-12	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-03 Listar boleto para reventa		
Actor Principal	Usuario		
Descripción	El sistema debe permitir cancelar la publicación de un boleto listado en reventa.		
Entradas	Boleto listado y solicitud de cancelación del propietario.		
Proceso	El sistema verifica que el usuario sea propietario del boleto y retira el boleto del mercado P2P.		
Salidas	Boleto eliminado del mercado de reventa.		
Razón	El usuario debe poder retirar su boleto si decide no venderlo.		
Criterio de medición	El boleto deja de aparecer en Reventa P2P y cambia su estado a no disponible para venta.		
Prioridad	Media	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 18: Documentación de Requerimientos RF-12

# Requerimiento	RF-13	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica		
Actor Principal	Usuario		
Descripción	El sistema debe permitir consultar los boletos disponibles en el mercado de reventa P2P.		
Entradas	Solicitud de consulta del mercado P2P.		
Proceso	El sistema obtiene los boletos marcados como disponibles para reventa.		
Salidas	Lista de boletos en reventa con evento, precio y disponibilidad.		
Razón	El comprador necesita visualizar las opciones disponibles antes de comprar.		
Criterio de medición	Solo se muestran boletos activos y disponibles para reventa.		
Prioridad	Alta	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 19: Documentación de Requerimientos RF-13

# Requerimiento	RF-14	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica		
Actor Principal	Usuario		
Descripción	El sistema debe permitir comprar un boleto publicado por otro usuario en el mercado de reventa P2P mediante Freightier.		
Entradas	Boleto en reventa, wallet del comprador y firma de transacción.		
Proceso	El sistema ejecuta la compra en blockchain, actualiza la propiedad del boleto y registra la operación.		
Salidas	Boleto transferido al comprador y operación registrada.		
Razón	Permite realizar compras entre usuarios de forma segura y trazable.		
Criterio de medición	El boleto aparece en Mis Entradas del comprador y desaparece del mercado P2P.		
Prioridad	Alta	Módulo Asociado	Reventa P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 20: Documentación de Requerimientos RF-14

# Requerimiento	RF-15	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica incluye Burn/Remint versión		
Actor Principal	Sistema / Contrato Soroban		
Descripción	El sistema debe invalidar la versión anterior del boleto y generar una nueva versión después de una reventa exitosa.		
Entradas	Identificador del boleto, versión actual, wallet del vendedor y wallet del comprador.		
Proceso	El contrato quema o invalida la versión anterior y genera una nueva versión asociada al nuevo propietario.		
Salidas	Nueva versión del boleto y QR anterior invalidado.		
Razón	Evita que el vendedor conserve un boleto válido después de la reventa.		
Criterio de medición	El QR antiguo es rechazado y el nuevo boleto pertenece al comprador.		
Prioridad	Alta	Módulo Asociado	Blockchain y NFTs
Versión	2.0	Fecha	[19/05/2026]

Cuadro 21: Documentación de Requerimientos RF-15

# Requerimiento	RF-16	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica incluye Distribuir comisiones		
Actor Principal	Sistema / Contrato Soroban		
Descripción	El sistema debe distribuir las comisiones generadas durante una compra en reventa P2P.		
Entradas	Precio de reventa, wallet del comprador, wallet del vendedor y reglas de comisión.		
Proceso	El contrato calcula los montos correspondientes y ejecuta la distribución según las reglas definidas.		
Salidas	Pago distribuido y comisión registrada.		
Razón	Permite controlar la monetización de la reventa y mantener reglas transparentes.		
Criterio de medición	La operación registra monto, comisión y wallets involucradas.		
Prioridad	Alta	Módulo Asociado	Reventa P2P / Contrato Soroban
Versión	2.0	Fecha	[19/05/2026]

Cuadro 22: Documentación de Requerimientos RF-16

# Requerimiento	RF-17	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-04 Compra en reventa atómica		
Actor Principal	Usuario		
Descripción	El sistema debe permitir consultar el historial de boletos vendidos en la sección Mis Ventas P2P.		
Entradas	Solicitud del usuario autenticado.		
Proceso	El sistema consulta las ventas asociadas al usuario vendedor.		
Salidas	Historial de ventas con monto, boleto vendido y wallet del comprador.		
Razón	El vendedor necesita llevar trazabilidad de sus operaciones de reventa.		
Criterio de medición	El usuario visualiza únicamente sus ventas realizadas.		
Prioridad	Media	Módulo Asociado	Mis Ventas P2P
Versión	2.0	Fecha	[19/05/2026]

Cuadro 23: Documentación de Requerimientos RF-17

# Requerimiento	RF-18	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-05 Verificar y redimir boleto en puerta		
Actor Principal	Verificador / Personal Evento		
Descripción	El sistema debe generar un código QR único para cada boleto válido.		
Entradas	Información del boleto, identificador y versión.		
Proceso	El sistema genera un QR asociado al boleto y a su versión vigente.		
Salidas	Código QR disponible para validación en puerta.		
Razón	El QR permite verificar la autenticidad del boleto durante el ingreso al evento.		
Criterio de medición	Cada boleto válido cuenta con un QR único y legible.		
Prioridad	Alta	Módulo Asociado	Validación QR
Versión	2.0	Fecha	[19/05/2026]

Cuadro 24: Documentación de Requerimientos RF-18

# Requerimiento	RF-19	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-05 Verificar y redimir boleto en puerta		
Actor Principal	Verificador / Personal Evento		
Descripción	El sistema debe permitir escanear el código QR de un boleto en la entrada del evento.		
Entradas	Código QR presentado por el usuario.		
Proceso	El sistema lee el QR, consulta el boleto asociado y valida su estado.		
Salidas	Resultado de validación del boleto.		
Razón	El personal del evento necesita verificar rápidamente la entrada del asistente.		
Criterio de medición	El QR puede ser leído correctamente desde la interfaz de escáner.		
Prioridad	Alta	Módulo Asociado	Escáner QR
Versión	2.0	Fecha	[19/05/2026]

Cuadro 25: Documentación de Requerimientos RF-19

# Requerimiento	RF-20	Tipo de Requerimiento	Funcional
Caso de Uso Asociado	CU-05 Verificar y redimir boleto en puerta		
Actor Principal	Verificador / Personal Evento		
Descripción	El sistema debe rechazar boletos usados, invalidados o con versiones vencidas por reventa.		
Entradas	Código QR y estado del boleto.		
Proceso	El sistema compara la versión del QR con la versión vigente y revisa si el boleto ya fue usado o invalidado.		
Salidas	Boleto aceptado o rechazado.		
Razón	Evita fraudes, duplicados y uso de boletos antiguos.		
Criterio de medición	Un QR antiguo, usado o invalidado es rechazado por el sistema.		
Prioridad	Alta	Módulo Asociado	Validación QR
Versión	2.0	Fecha	[19/05/2026]

Cuadro 26: Documentación de Requerimientos RF-20

Los requerimientos funcionales anteriores se encuentran relacionados directamente con los casos de uso significativos del sistema Stellar Tickets, esta trazabilidad permite identificar qué funcionalidad cumple cada caso de uso, qué actor interviene, qué información ingresa al sistema, qué proceso se ejecuta y qué resultado se espera y también facilita el mantenimiento del sistema, ya que cualquier cambio en un caso de uso puede reflejarse en los requerimientos funcionales asociados.

3.1.1. Funcionalidad 1: Gestión de eventos y contratos

Esta funcionalidad reúne los elementos necesarios para que el administrador pueda crear eventos dentro de la plataforma y asociarlos con contratos desplegados en Soroban. Aquí se documentan los datos principales del evento y la información técnica del contrato que permite administrar boletos en blockchain.

ID	MD-01	
Elemento del Dominio	Evento y Contrato Soroban	
Requerimientos asociados	RF-01 Crear evento y desplegar contrato Soroban	
Casos de uso asociados	CU-01 Crear evento y desplegar contrato	
Descripción	Representa la información base de un evento publicado en la plataforma y su relación con un contrato inteligente desplegado en Stellar Testnet. Este elemento permite que cada evento tenga una estructura propia para manejar boletos, ventas, reventas e invalidaciones.	
Atributos		
Nombre	Descripción	Tipo de Dato
event_id	Identificador único del evento dentro del sistema.	UUID / VARCHAR(36)
title	Nombre visible del evento para los usuarios.	VARCHAR(255)
event_date	Fecha y hora programada del evento.	TIMESTAMP WITH TIME ZONE
venue_id	Identificador del lugar donde se realizará el evento.	UUID / FOREIGN KEY
capacity	Número máximo de asistentes permitidos para el evento.	INTEGER
image_url	Ruta o URL de la imagen promocional del evento.	TEXT / URL
category	Categoría del evento, por ejemplo concierto, deporte o teatro.	ENUM / VARCHAR(50)
city	Ciudad donde se realizará el evento.	VARCHAR(100)
contract_address	Dirección del contrato desplegado en Soroban asociado al evento.	Stellar Contract Address / VARCHAR(100)
contract_status	Estado del contrato asociado al evento.	ENUM
created_by	Usuario administrador que creó el evento.	UUID / FOREIGN KEY
created_at	Fecha de creación del registro.	TIMESTAMP WITH TIME ZONE
Objetivo	Permitir que la plataforma registre eventos administrables y los conecte con contratos blockchain para soportar la emisión, venta, reventa e invalidación de boletos digitales.	

Cuadro 27: Formato de documentación del Modelo del Dominio

3.1.2. Funcionalidad 2: Compra primaria de boletos

Esta funcionalidad describe los elementos que intervienen cuando un usuario consulta eventos, selecciona boletos, administra su carrito y realiza una compra primaria. Su propósito es permitir que el cliente adquiera entradas directamente desde la plataforma antes de cualquier proceso de reventa.

ID	MD-02	
Elemento del Dominio	Catálogo, Carrito, Orden de Compra y Boleto	
Requerimientos asociados	RF-02 Comprar boleto en venta primaria	
Casos de uso asociados	CU-02 Compra primaria de boleto	
Descripción	Representa el conjunto de datos que permite al usuario buscar eventos, seleccionar entradas, agregarlas al carrito y finalizar una compra mediante pago simulado. También incluye la visualización posterior de los boletos adquiridos en la sección Mis Entradas.	
Atributos		
Nombre	Descripción	Tipo de Dato
catalog_filter	Criterios usados para filtrar eventos por ciudad o categoría.	JSONB / OBJECT
cart_id	Identificador único del carrito del usuario.	UUID
user_id	Identificador del usuario dueño del carrito o de la orden.	UUID / FOREIGN KEY
event_id	Evento al que pertenece el boleto seleccionado.	UUID / FOREIGN KEY
ticket_type	Tipo de boleto seleccionado, general o con asiento.	ENUM('GA','SEATED')
seat_id	Identificador del asiento seleccionado, cuando aplica.	UUID / NULLABLE FOREIGN KEY
quantity	Cantidad de boletos seleccionados.	INTEGER
unit_price	Precio unitario del boleto.	DECIMAL(12,2)
order_id	Identificador de la orden generada después del checkout.	UUID
payment_status	Estado del pago simulado.	ENUM
ticket_status	Estado inicial del boleto adquirido.	ENUM
created_at	Fecha en que se generó el carrito, orden o boleto.	TIMESTAMP WITH TIME ZONE
Objetivo	Facilitar el proceso completo de compra primaria, desde la consulta del evento hasta la generación de boletos asociados al usuario autenticado.	

Cuadro 28: Formato de documentación del Modelo del Dominio

3.1.3. Funcionalidad 3: Wallets e integración Web3

Esta funcionalidad contempla la conexión entre la cuenta tradicional del usuario y su wallet Freightier. Es importante porque algunas operaciones del sistema, especialmente las relacionadas con blockchain, requieren identificar la dirección pública del usuario y permitir la firma de transacciones.

ID	MD-03	
Elemento del Dominio	Usuario Web3 y Wallet Freighter	
Requerimientos asociados	RF-09 Conectar wallet Freighter, RF-10 Asegurar boleto en blockchain	
Casos de uso asociados	CU-03 Listar boleto para reventa	
Descripción	Representa la relación entre una cuenta de usuario Web2 y una dirección pública de Stellar. Esta información permite habilitar funcionalidades como asegurar boletos en blockchain, consultar propiedad y firmar operaciones de reventa mediante Freighter.	
Atributos		
Nombre	Descripción	Tipo de Dato
user_id	Identificador del usuario registrado en la plataforma.	UUID / FOREIGN KEY
name	Nombre del usuario.	VARCHAR(150)
email	Correo electrónico usado para autenticación.	VARCHAR(255) / UNIQUE
password_hash	Contraseña cifrada del usuario.	VARCHAR(255)
role	Rol asignado al usuario dentro del sistema.	ENUM
wallet_address	Dirección pública Stellar vinculada a la cuenta.	Stellar Public Key / VARCHAR(56)
wallet_provider	Proveedor de wallet usado por el usuario.	ENUM('Freighter')
wallet_linked_at	Fecha en la que la wallet fue vinculada.	TIMESTAMP WITH TIME ZONE
is_wallet_verified	Indica si la wallet fue conectada correctamente.	BOOLEAN
Objetivo	Asociar de manera controlada una cuenta de usuario con una wallet Web3, permitiendo ejecutar operaciones blockchain únicamente cuando el usuario tenga una dirección válida vinculada.	

Cuadro 29: Formato de documentación del Modelo del Dominio

3.1.4. Funcionalidad 4: Blockchain y NFTs

Esta funcionalidad agrupa los elementos que permiten representar boletos digitales como activos verificables en blockchain. Incluye la información del NFT, la versión del boleto y los datos necesarios para conservar trazabilidad entre la base de datos y Stellar Testnet.

ID	MD-04	
Elemento del Dominio	Boleto Blockchain y NFT	
Requerimientos asociados	RF-10 Asegurar boleto en blockchain, RF-06 y RF-15 Burn/Remint de versión	
Casos de uso asociados	CU-03 Listar boleto para reventa, CU-04 Compra en reventa atómica	
Descripción	Representa el boleto cuando ya ha sido asegurado en blockchain y cuenta con una versión NFT asociada. Este elemento permite mantener la propiedad, el estado y la trazabilidad del boleto durante procesos de compra, reventa o invalidación.	
Atributos		
Nombre	Descripción	Tipo de Dato
ticket_id	Identificador interno del boleto en la base de datos.	UUID
contract_address	Dirección del contrato Soroban asociado al evento.	Stellar Contract Address / VARCHAR(100)
ticket_root_id	Identificador raíz del boleto, usado para conservar trazabilidad entre versiones.	VARCHAR(100)
version	Número de versión del boleto.	INTEGER
nft_token_id	Identificador del NFT asociado al boleto.	VARCHAR(100)
token_uri	URL de metadata del NFT.	TEXT / URL
owner_wallet	Wallet propietaria actual del boleto.	Stellar Public Key / VARCHAR(56)
asset_code	Código del activo relacionado con el boleto.	VARCHAR(20)
blockchain_status	Estado del boleto en blockchain.	ENUM
tx_hash	Hash de la transacción blockchain relacionada.	VARCHAR(128)
synced_at	Fecha de última sincronización con el indexer.	TIMESTAMP WITH TIME ZONE
Objetivo	Garantizar que cada boleto asegurado tenga una representación digital única, verificable y trazable en blockchain, evitando duplicidades o usos no autorizados.	

Cuadro 30: Formato de documentación del Modelo del Dominio

3.1.5. Funcionalidad 5: Reventa P2P

Esta funcionalidad permite que los usuarios publiquen boletos propios en un mercado secundario y que otros usuarios puedan comprarlos mediante Freightier. También contempla la actualización de propiedad, el historial de ventas y la distribución de comisiones asociadas a la operación.

ID	MD-05	
Elemento del Dominio	Publicación de Reventa y Transacción P2P	
Requerimientos asociados	RF-03 Listar boleto para reventa, RF-04 Comprar boleto en reventa atómica, RF-05 Distribuir comisiones, RF-06 Burn/Remint de versión, RF-11 Publicar boleto en reventa, RF-12 Cancelar publicación, RF-13 Consultar boletos en reventa, RF-14 Comprar boleto en reventa, RF-16 Distribuir comisiones, RF-17 Historial de ventas P2P	
Casos de uso asociados	CU-03 Listar boleto para reventa, CU-04 Compra en reventa atómica	
Descripción	Representa el proceso mediante el cual un boleto pasa del propietario original a un nuevo comprador dentro del mercado P2P. Este elemento conserva información del precio, vendedor, comprador, comisión, estado de la publicación y resultado de la transferencia.	
Atributos		
Nombre	Descripción	Tipo de Dato
resale_id	Identificador único de la publicación de reventa.	UUID
ticket_id	Boleto que será ofrecido en el mercado P2P.	UUID / FOREIGN KEY
seller_user_id	Usuario que publica el boleto para reventa.	UUID / FOREIGN KEY
seller_wallet	Wallet del vendedor del boleto.	Stellar Public Key / VARCHAR(56)
buyer_user_id	Usuario que compra el boleto, cuando la operación se concreta.	UUID / NULLABLE FOREIGN KEY
buyer_wallet	Wallet del comprador del boleto.	Stellar Public Key / VARCHAR(56) / NULLABLE
resale_price	Precio definido por el vendedor para la reventa.	DECIMAL(18,7)
commission_amount	Valor de la comisión generada por la operación.	DECIMAL(18,7)
resale_status	Estado de la publicación o transacción.	ENUM
signed_xdr	Transacción firmada por el comprador mediante Freighter.	TEXT / XDR String
tx_hash	Hash de la transacción de compra en reventa.	VARCHAR(128)
sold_at	Fecha en la que se concreta la venta.	TIMESTAMP WITH TIME ZONE / NULLABLE
Objetivo	Permitir un mercado secundario controlado, donde los boletos puedan revenderse de forma trazable, actualizando la propiedad y evitando que versiones anteriores sigan siendo válidas.	

Cuadro 31: Formato de documentación del Modelo del Dominio

3.1.6. Funcionalidad 6: Validación QR

Esta funcionalidad se enfoca en el proceso de ingreso al evento. Permite que el personal autorizado escanee el código QR del boleto, verifique su estado y lo marque como usado si cumple con las condiciones de validez.

ID	MD-06	
Elemento del Dominio	Código QR y Validación de Boleto	
Requerimientos asociados	RF-18 Generar QR, RF-19 Escanear QR, RF-20 Rechazar boletos usados, invalidados o vencidos	
Casos de uso asociados	CU-05 Verificar y redimir boleto en puerta	
Descripción	Representa la información utilizada para validar un boleto en la entrada del evento. El QR contiene datos del boleto y su versión vigente, permitiendo detectar si el boleto ya fue usado, invalidado o reemplazado después de una reventa.	
Atributos		
Nombre	Descripción	Tipo de Dato
qr_id	Identificador del código QR generado para el boleto.	UUID
ticket_id	Boleto asociado al código QR.	UUID / FOREIGN KEY
ticket_root_id	Identificador raíz del boleto.	VARCHAR(100)
version	Versión del boleto incluida en el QR.	INTEGER
qr_payload	Información codificada dentro del QR.	JSONB / QRPayload
qr_url	URL usada para consultar o validar el QR.	TEXT / URL
scanned_by	Usuario STAFF o ADMIN que realiza el escaneo.	UUID / FOREIGN KEY
scan_status	Resultado del proceso de validación.	ENUM
scanned_at	Fecha y hora del escaneo.	TIMESTAMP WITH TIME ZONE
http_response_code	Código de respuesta generado por el sistema durante la validación.	INTEGER
Objetivo	Controlar el ingreso al evento mediante una validación rápida y segura, garantizando que solo boletos vigentes, no usados y no invalidados puedan ser redimidos en puerta.	

Cuadro 32: Formato de documentación del Modelo del Dominio

3.1.7. Funcionalidad 7: Administración y control de boletos

Esta funcionalidad permite que actores autorizados, como la ticketera u organizador, puedan tomar acciones sobre boletos específicos en casos especiales. Se usa cuando un boleto debe ser bloqueado por fraude, error, cancelación o alguna situación que comprometa la validez de la entrada.

ID	MD-07	
Elemento del Dominio	Invalidación de Boleto	
Requerimientos asociados	RF-08 Invalidar boleto	
Casos de uso asociados	CU-06 Invalidar boleto	
Descripción	Representa el proceso mediante el cual un boleto deja de ser válido dentro del sistema. La invalidación impide que el boleto pueda ser revendido, transferido o validado en puerta, manteniendo un registro del motivo y del actor que realizó la acción.	
Atributos		
Nombre	Descripción	Tipo de Dato
invalidation_id	Identificador único del registro de invalidación.	UUID
ticket_id	Boleto que será invalidado.	UUID / FOREIGN KEY
ticket_root_id	Identificador raíz del boleto afectado.	VARCHAR(100)
invalidated_by	Usuario organizador o administrador que ejecuta la acción.	UUID / FOREIGN KEY
invalidation_reason	Motivo por el cual se invalida el boleto.	TEXT
previous_status	Estado del boleto antes de ser invalidado.	ENUM
new_status	Nuevo estado asignado al boleto.	ENUM('invalidated')
invalidated_at	Fecha y hora de la invalidación.	TIMESTAMP WITH TIME ZONE
Objetivo	Permitir que la plataforma mantenga control sobre situaciones excepcionales, protegiendo la autenticidad del sistema y evitando el uso indebido de boletos comprometidos.	

Cuadro 33: Formato de documentación del Modelo del Dominio

3.1.8. Funcionalidad 8: Cruce de funcionalidades

En esta sección se incluyen elementos que no pertenecen a una sola funcionalidad, pero que son necesarios para el correcto funcionamiento de varias partes del sistema. Estos elementos permiten mantener trazabilidad, control de acceso, sincronización con blockchain y coherencia entre los módulos.

ID	MD-08	
Elemento del Dominio	Autenticación, Roles e Indexer	
Requerimientos asociados	Soporte transversal para RF-09, RF-10, RF-14, RF-15, RF-16, RF-19 y RF-20; los controles de registro, sesión y roles se documentan como restricciones de seguridad y diseño, no como RF adicionales.	
Casos de uso asociados	Transversal a CU-01, CU-02, CU-03, CU-04, CU-05 y CU-06	
Descripción	Agrupa los elementos que permiten que el sistema controle el acceso de los usuarios, mantenga sesiones activas y sincronice información entre blockchain y base de datos. Aunque no representan una funcionalidad visible única, son necesarios para garantizar que las acciones se ejecuten con permisos correctos y datos actualizados.	
Atributos		
Nombre	Descripción	Tipo de Dato
auth_token	Token usado para mantener la sesión del usuario autenticado.	JWT String
token_expiration	Tiempo de expiración del token de sesión.	INTERVAL / TIMESTAMP
role	Rol del usuario dentro del sistema.	ENUM
permission_scope	Alcance de permisos disponibles para el usuario.	JSONB / ARRAY
indexer_state_id	Identificador del estado del indexer.	UUID
last_processed_ledger	Último ledger procesado por el indexer.	BIGINT
sync_status	Estado de sincronización entre blockchain y base de datos.	ENUM
provider_reference	Referencia externa de una transacción o pago.	VARCHAR(128)
created_at	Fecha de creación del registro.	TIMESTAMP WITH TIME ZONE
updated_at	Fecha de última actualización del registro.	TIMESTAMP WITH TIME ZONE
Objetivo	Servir como soporte transversal para la seguridad, trazabilidad y sincronización del sistema, permitiendo que los diferentes módulos operen de forma coherente y controlada.	

Cuadro 34: Formato de documentación del Modelo del Dominio

3.2. Requerimientos de Desempeño

Los requerimientos de desempeño del sistema Stellar Tickets definen las condiciones mínimas bajo las cuales la aplicación debe funcionar de manera estable para los diferentes tipos de usuario, estos requerimientos consideran la cantidad de usuarios conectados al sistema, el acceso desde navegadores web en computador o móvil, las acciones principales que realizan los usuarios y los tiempos de respuesta esperados para operaciones como consulta de eventos, compra de boletos, reventa P2P y validación QR en puerta.

3.2.1. Requerimientos estáticos de desempeño

Los requerimientos estáticos hacen referencia a la capacidad que debe soportar el sistema en términos de usuarios, dispositivos y módulos disponibles al mismo tiempo.

ID	Requerimiento de desempeño	Tipo	Criterio de medición
RD-01	El sistema debe permitir el acceso simultáneo de usuarios tipo CUSTOMER, ADMIN y STAFF, cada uno con las funcionalidades correspondientes a su rol.	Estático	Durante una prueba de uso, cada rol debe visualizar únicamente las pantallas y acciones autorizadas sin afectar el acceso de los demás usuarios.
RD-02	El sistema debe soportar el acceso desde navegadores modernos en dispositivos de escritorio y móviles, sin requerir una aplicación nativa.	Estático	La aplicación debe cargar correctamente en navegadores web modernos y adaptarse a pantallas de computador y celular.
RD-03	El sistema debe permitir que usuarios CUSTOMER naveguen por el catálogo, consulten eventos, gestionen carrito, compren boletos y accedan a Mis Entradas.	Estático	Las pantallas principales del CUSTOMER deben estar disponibles durante la sesión del usuario autenticado.
RD-04	El sistema debe permitir que usuarios ADMIN accedan al panel administrativo para crear eventos y consultar contratos desplegados en Soroban.	Estático	El panel administrativo debe estar disponible únicamente para usuarios con rol ADMIN.
RD-05	El sistema debe permitir que usuarios STAFF accedan únicamente al escáner QR para validar boletos en puerta.	Estático	El usuario STAFF no debe tener acceso a funciones de compra, reventa o administración general.
RD-06	El sistema debe soportar múltiples dispositivos conectados al mismo tiempo, incluyendo computadores para administración y celulares para consulta o validación de boletos.	Estático	La aplicación debe mantener sesiones activas desde distintos dispositivos sin bloquear la operación general del sistema.

Cuadro 35: Requerimientos de desempeño

3.2.2. Requerimientos dinámicos de desempeño

Los requerimientos dinámicos se relacionan con el comportamiento del sistema cuando los usuarios realizan acciones específicas y para este caso, se deben considerar los tiempos de respuesta de las pantallas principales y la rapidez de las operaciones más importantes.

ID	Requerimiento de desempeño	Tipo	Criterio de medición
RD-07	El sistema debe cargar el catálogo de eventos en un tiempo máximo de 3 segundos bajo condiciones normales de conexión.	Dinámico	Al ingresar al catálogo, los eventos deben mostrarse en máximo 3 segundos.
RD-08	El sistema debe aplicar filtros por categoría o ciudad en un tiempo máximo de 2 segundos .	Dinámico	Al seleccionar un filtro, la lista de eventos debe actualizarse en máximo 2 segundos.
RD-09	El sistema debe cargar el detalle de un evento y mostrar la disponibilidad de boletos generales o con asiento en un tiempo máximo de 3 segundos .	Dinámico	La información del evento y sus boletos debe estar visible antes de superar el tiempo definido.
RD-10	El sistema debe actualizar el carrito de compras en un tiempo máximo de 2 segundos después de agregar, modificar o eliminar boletos.	Dinámico	Cada acción sobre el carrito debe reflejarse correctamente en la interfaz.
RD-11	El sistema debe procesar el checkout con pago simulado en un tiempo máximo de 5 segundos , siempre que los servicios necesarios estén disponibles.	Dinámico	Al confirmar la compra, el sistema debe generar la orden y asignar los boletos dentro del tiempo establecido.
RD-12	El sistema debe cargar la sección Mis Entradas en un tiempo máximo de 3 segundos .	Dinámico	El usuario debe visualizar sus boletos adquiridos sin superar el tiempo definido.
RD-13	El sistema debe permitir conectar la wallet Freightier en un tiempo razonable, dependiendo de la autorización del usuario desde la extensión.	Dinámico	Una vez el usuario apruebe la conexión en Freightier, la wallet debe quedar vinculada a su cuenta.
RD-14	El sistema debe publicar un boleto en el mercado de Reventa P2P en un tiempo máximo de 5 segundos , excluyendo tiempos externos de confirmación Web3 cuando apliquen.	Dinámico	El boleto debe cambiar su estado a “en venta” y aparecer en el módulo de Reventa P2P.
RD-15	El sistema debe cargar los boletos disponibles en Reventa P2P en un tiempo máximo de 3 segundos .	Dinámico	La pantalla de reventa debe mostrar únicamente boletos activos y disponibles.
RD-16	El sistema debe registrar la compra de un boleto en reventa después de la firma del usuario en Freightier, considerando que la confirmación puede depender de servicios blockchain externos.	Dinámico	Después de firmar, el boleto debe actualizarse y aparecer asociado al nuevo comprador.
RD-17	El sistema debe cargar el historial de Mis Ventas P2P en un tiempo máximo de 3 segundos .	Dinámico	El usuario vendedor debe poder consultar sus ventas con monto y wallet del comprador.
RD-18	El sistema debe validar un código QR en puerta en un tiempo máximo de 2 segundos bajo condiciones normales.	Dinámico	Al escanear el QR, el sistema debe responder rápidamente si el boleto es válido, usado, invalidado o vencido.
RD-19	El sistema debe rechazar de forma inmediata los boletos con QR vencido, usado o invalidado.	Dinámico	El sistema debe mostrar una respuesta de rechazo y evitar que el boleto sea redimido.
RD-20	El sistema debe mantener una navegación fluida entre pantallas principales de la SPA, evitando recargas completas innecesarias.	Dinámico	La transición entre catálogo, detalle, carrito, Mis Entradas y Reventa P2P debe realizarse desde la interfaz web sin pérdida de sesión.

Cuadro 36: Requerimientos de desempeño dinámicos

3.3. Restricciones de Diseño

El diseño del sistema Stellar Tickets se encuentra condicionado por decisiones tecnológicas, arquitectónicas y de despliegue previamente definidas en el documento de diseño de software, estas restricciones deben ser tenidas en cuenta durante la implementación, ya que determinan

los lenguajes, herramientas, arquitectura, infraestructura y modelo de datos que se utilizarán para construir la solución.

3.3.1. Lenguajes de programación

El sistema debe desarrollarse utilizando los lenguajes definidos para cada capa de la arquitectura. La capa de presentación debe implementarse con TypeScript, utilizando React 18 y Vite para construir una aplicación web SPA, la capa de servicios también debe usar TypeScript, ejecutándose sobre Node.js con Express. Ahora, la capa blockchain debe desarrollarse en Rust, ya que los contratos inteligentes de Soroban se compilan a WebAssembly (WASM). Esta restricción se debe a que el documento de arquitectura define el uso de Stellar/Soroban como plataforma blockchain y establece que los contratos deben escribirse en Rust y compilarse a WASM.

3.3.2. Herramientas de análisis, diseño e implementación

Para el análisis y diseño del sistema se debe mantener la documentación arquitectónica bajo el enfoque del modelo C4, utilizado para representar el contexto y los contenedores del sistema. El SAD indica que este modelo permite comunicar la arquitectura a diferentes niveles de detalle y es adecuado para un sistema híbrido con componentes *on-chain* y *off-chain*.

En la implementación, el proyecto debe organizarse como un monorepo que contiene los contratos Soroban, el backend y el frontend. La estructura documentada incluye una carpeta `contracts/` para los contratos en Rust, una carpeta `backend/` para el servidor Express con Prisma e indexador Soroban, y una carpeta `frontend/` para la aplicación React con TypeScript y Freighter.

3.3.3. Restricciones de Hardware y Software

El sistema no requiere hardware propio, servidores físicos ni máquinas virtuales administradas directamente por el equipo. Su despliegue debe realizarse sobre servicios cloud y, según la vista de despliegue del SAD, el cliente utiliza un navegador con Freighter, el frontend se sirve desde Vercel, el backend e indexador se ejecutan en Railway o Render, la base de datos está en Supabase con PostgreSQL 16 y la capa blockchain opera sobre Stellar Testnet con Soroban.

Una restricción importante es que el indexador Soroban requiere un proceso persistente, por lo que debe ejecutarse en un entorno *long-lived* como Railway o Render. Este no debe ejecutarse en un entorno *serverless* como Vercel, ya que allí se desactiva el `app.listen()` y el indexador.

Además, el sistema depende de variables de entorno obligatorias para operar correctamente, como `DATABASE_URL`, `JWT_SECRET`, `SOROBAN_RPC_URL`, `ORGANIZER_SECRET`, `FACTORY_CONTRACT_ID` y `VITE_API_BASE_URL`. En producción, `JWT_SECRET` es obligatorio y el servidor no debe arrancar si no está definido.

3.3.4. Restricciones de arquitectura

El sistema debe seguir una arquitectura por capas, separando claramente la presentación, los servicios, los datos y la blockchain, teniendo en cuenta el documento SAD define cuatro capas principales, la capa de presentación, encargada de la interfaz de usuario, la capa de servicios, encargada de la lógica off-chain, autenticación y endpoints, la capa de datos, basada en PostgreSQL para consultas rápidas, auditoría e indexación y la capa blockchain, responsable de la lógica on-chain y la fuente de verdad para propiedad de tickets y pagos.

También se debe mantener la separación entre lógica on-chain y off-chain y el principio de diseño documentado indica que lo relacionado con propiedad o dinero debe ejecutarse on-chain, mientras que lo que requiere consultas rápidas o contexto de negocio debe resolverse off-chain.

3.3.5. Restricciones de Blockchain y contratos inteligentes

El sistema debe operar sobre Stellar Testnet y utilizar contratos inteligentes Soroban. Al tratarse de un prototipo académico, la red utilizada no maneja activos con valor monetario real y puede presentar limitaciones propias de una red de prueba.

El diseño de contratos debe mantener el patrón *Factory*, donde un contrato *FabricaBoletos* despliega un contrato independiente por cada evento, lo que permite aislar la configuración, el almacenamiento y los fondos de cada evento, como se muestra en el SAD. También se establece que la compra en reventa debe aplicar el patrón *burn/remint*, invalidando la versión anterior del boleto y creando una nueva versión para el comprador.

Adicionalmente, las operaciones que modifican estado en los contratos deben validar autorización mediante `require_auth()`, esto aplica para acciones como crear boletos, listar boletos, cancelar ventas, comprar boletos, redimir e invalidar boletos. Además, las reglas de integridad del contrato impiden, por ejemplo, revender boletos usados o invalidados, comprar el propio boleto o reutilizar versiones anteriores.

3.3.6. Restricciones de la base de datos

El modelo de datos debe construirse sobre PostgreSQL 16 en Supabase, utilizando el `schema ticketing`. La base de datos debe almacenar usuarios, eventos, boletos, órdenes, pagos, carrito, inventario de asientos, historial de compras e información Web3 sincronizada desde blockchain.

Los boletos deben conservar campos que permitan relacionar la base de datos con la blockchain, como `contract_address`, `ticket_root_id`, `version`, `is_for_sale`, `resale_price` y `owner_wallet`. Además, debe mantenerse una restricción única sobre la combinación `contract_address`, `ticket_root_id` y `version`, ya que esta permite identificar de forma precisa cada versión de un boleto.

La base de datos no reemplaza la blockchain como fuente de verdad para propiedad y pagos. Su función principal es facilitar consultas rápidas, auditoría, indexación y experiencia de usuario. Adicionalmente, la sincronización con blockchain debe realizarse mediante el indexador Soroban, que actualiza PostgreSQL a partir de los eventos *on-chain*.

3.3.7. Restricciones de autenticacion y seguridad

La autenticación debe implementarse mediante JWT y contraseñas cifradas con bcrypt. El backend debe validar los roles de usuario en las rutas protegidas, especialmente en las funciones administrativas y en el escáner QR y el documento SAD establece que el backend utiliza JWT, bcrypt y verificación de roles para endpoints de administración y escaneo.

El sistema debe manejar tres roles principales CUSTOMER, ADMIN y STAFF, el rol CUSTOMER accede a compra, entradas y reventa, el ADMIN accede a gestión de eventos, contratos y validación, el STAFF solo accede al escáner QR, con esta restricción condiciona el diseño de rutas, permisos, vistas y validaciones del backend.

3.3.8. Restricciones de despliegue

El frontend debe desplegarse como una *SPA* en Vercel, mientras que el backend principal y el indexador deben desplegarse en Railway o Render. La base de datos debe permanecer en Supabase y las operaciones blockchain deben ejecutarse sobre Stellar Testnet. El SAD presenta esta topología de despliegue como una estructura *cloud* con cliente, frontend, backend/indexador, base de datos y blockchain.

En desarrollo local, el frontend debe ejecutarse con `npm run dev` usando Vite, y el backend con `npm run dev` sobre `tsx watch`. Esto permite mantener separados los entornos de desarrollo y producción, respetando la estructura definida en el documento de diseño.

3.4. Atributos del Sistema de Software (No funcionales)

3.4.1. Seguridad

- Descripción: Garantizar que cada usuario solo pueda realizar operaciones autorizadas según su rol (CUSTOMER, ADMIN, STAFF), que las transacciones de compra y reventa sean atómicas y que los boletos no puedan falsificarse ni duplicarse.
- Criterios de medición: Validación de roles en cada endpoint; pruebas de acceso indebido; verificación de firmas on-chain para transacciones Soroban.
- Prioridad: Alta

# Requerimiento	RNF-01	Tipo de Requerimiento	No funcional
Casos de Uso Asociados	CU-02, CU-03, CU-04, CU-05, CU-06		
Descripción	Garantizar que las operaciones del sistema se realicen de forma segura: autenticación de usuarios, permisos por rol y firma de transacciones Web3 mediante Freightier.		
Razón	Protege la información de los usuarios y previene accesos no autorizados o fraudes en compra, reventa y redención de boletos.		
Autor	Equipo de desarrollo		
Criterio de medición	Todas las rutas protegidas verifican JWT y roles; pruebas de intento de acceso indebido fallan; las transacciones Web3 requieren firma válida.		
Prioridad	Alta	Módulo Asociado	Seguridad / Autenticación
Versión	2.0	Fecha	[19/05/2026]

Cuadro 37: Documentación de Requerimientos No Funcionales RNF-01

3.4.2. Integridad

- Descripción: Las transacciones de compra y reventa deben ejecutarse de forma atómica, evitando estados inconsistentes donde un pago se procese pero el boleto no se transfiera, o viceversa.
- Criterios de medición: Simulación de transacciones concurrentes; revisión de logs de blockchain y base de datos; chequeo de consistencia entre off-chain y on-chain.
- Prioridad: Alta

# Requerimiento	RNF-02	Tipo de Requerimiento	No funcional
Casos de Uso Asociados	CU-02, CU-04		
Descripción	Mantener la consistencia de la información entre la base de datos y blockchain, evitando inconsistencias en compras, reventas o redenciones.		
Razón	Garantiza que las transacciones sean confiables y los datos reflejen la realidad de la propiedad de boletos.		
Autor	Equipo de desarrollo		
Criterio de medición	Transacciones completadas correctamente reflejan cambios consistentes en DB y blockchain; no se generan duplicados ni pérdidas de datos.		
Prioridad	Alta	Módulo Asociado	Blockchain / Base de datos
Versión	2.0	Fecha	[19/05/2026]

Cuadro 38: Documentación de Requerimientos No Funcionales RNF-02

3.4.3. Trazabilidad

- Descripción: Mantener historial completo de versiones de cada boleto, incluyendo transferencias, reventas y cambios de estado. Esto permite auditar la propiedad y la procedencia de cada ticket.
- Criterios de medición: Cada versión del boleto debe estar registrada y disponible para consulta; pruebas de reconstrucción de la cadena de propiedad desde cualquier versión.

- Prioridad: Alta

# Requerimiento	RNF-03	Tipo de Requerimiento	No funcional
Casos de Uso Asociados	CU-01, CU-02, CU-03, CU-04, CU-05		
Descripción	La plataforma debe permanecer accesible para los usuarios y personal STAFF incluso durante picos de tráfico y latencia de servicios externos.		
Razón	Garantiza que los usuarios puedan realizar consultas, compras y validaciones sin interrupciones.		
Autor	Equipo de desarrollo		
Criterio de medición	Disponibilidad mínima del 99 % en pruebas de carga; consultas al catálogo y redención de boletos responden según los tiempos definidos.		
Prioridad	Alta	Módulo Asociado	Frontend / Backend / Indexer
Versión	1.0	Fecha	[19/05/2026]

Cuadro 39: Documentación de Requerimientos No Funcionales RNF-03

3.4.4. Modificabilidad

- Descripción: Separación clara entre lógica on-chain (Soroban) y off-chain (PostgreSQL), permitiendo actualizar la base de datos o la interfaz sin afectar la integridad de los contratos.
- Criterios de medición: Simulación de cambios en la base de datos o en el frontend sin interrumpir transacciones en blockchain; pruebas de despliegue incremental.
- Prioridad: Medio

# Requerimiento	RNF-04	Tipo de Requerimiento	No funcional
Casos de Uso Asociados	CU-03, CU-04, CU-05		
Descripción	Mantener registro completo de la propiedad de boletos, incluyendo cambios de versión, reventas y transacciones realizadas por cada usuario.		
Razón	Permite auditar la propiedad y asegurar que los boletos no sean duplicados ni utilizados indebidamente.		
Autor	Equipo de desarrollo		
Criterio de medición	Todos los boletos tienen historial disponible y se puede reconstruir la cadena de propiedad.		
Prioridad	Alta	Módulo Asociado	Base de datos / Blockchain
Versión	1.0	Fecha	[19/05/2026]

Cuadro 40: Documentación de Requerimientos No Funcionales RNF-04

3.4.5. Disponibilidad

- Descripción: El backend debe ser resiliente; consultas de catálogo, Mis Entradas y Reventa P2P deben estar disponibles aun si alguna parte de la red blockchain tiene latencia.

- Criterios de medición: Pruebas de carga con fallos simulados de RPC Soroban o de la base de datos; porcentaje de tiempo en que los servicios críticos permanecen operativos.
- Prioridad: Alta

# Requerimiento	RNF-05	Tipo de Requerimiento	No funcional
Casos de Uso Asociados	CU-02, CU-03, CU-04, CU-05		
Descripción	Las operaciones críticas del sistema (consultar catálogo, carrito, checkout, reventa, validación QR) deben completarse dentro de los tiempos de respuesta establecidos.		
Razón	Mejora la experiencia de usuario y garantiza flujo eficiente en compra y acceso a eventos.		
Autor	Equipo de desarrollo		
Criterio de medición	Catálogo y filtros <3 seg; Checkout <5 seg; QR <2 seg; Reventa P2P <5 seg.		
Prioridad	Alta	Módulo Asociado	Frontend / Backend / Indexer
Versión	1.0	Fecha	[19/05/2026]

Cuadro 41: Documentación de Requerimientos No Funcionales RNF-05

3.5. Requerimientos de la base de datos

3.5.1. Tipos de datos almacenados

Se deben definir los tipos de datos más usados según PostgreSQL (motor elegido en Supabase) para cada tabla.

Entidad	Campo	Tipo de dato	Comentario
Usuario	user_id	UUID	Primary key
Usuario	name	VARCHAR(150)	Nombre completo
Usuario	email	VARCHAR(255)	Único, usado para login
Usuario	password_hash	VARCHAR(255)	Contraseña cifrada
Usuario	role	ENUM('CUSTOMER','ADMIN','STAFF')	Rol del usuario
Evento	event_id	UUID	Primary key
Evento	title	VARCHAR(255)	Título del evento
Evento	event_date	TIMESTAMP WITH TIME ZONE	Fecha y hora
Evento	capacity	INTEGER	Número máximo de boletos
Boleto	ticket_root_id	VARCHAR(100)	Identificador raíz para blockchain
Boleto	version	INTEGER	Control de versiones
Boleto	owner_wallet	VARCHAR(56)	Wallet actual propietaria
Orden	order_id	UUID	Primary key
Orden	total	NUMERIC(12,2)	Monto total de la compra
Reventa P2P	resale_id	UUID	Primary key
Reventa P2P	resale_price	NUMERIC(12,2)	Precio de reventa

Cuadro 42: Modelo de datos: entidades y campos principales

3.5.2. Tipo de consultas utilizadas

Para proteger la base de datos y evitar inyecciones:

- Todas las consultas deben ejecutarse mediante Prepared Statements.
- Los DAO (Data Access Objects) se encargan de construir consultas parametrizadas.
- Las consultas frecuentes incluyen:

- Selección de eventos disponibles: `SELECT * FROM events WHERE ...`
- Inserción de órdenes y boletos: `INSERT INTO orders ...`
- Actualización de estado de boletos: `UPDATE tickets SET status=...`
- Consultas de reventa P2P: `SELECT * FROM resale WHERE status='listed'`

3.5.3. Indexación de los datos

Para optimizar consultas:

- Se deben crear índices en campos de búsqueda frecuente:
 - `event_id` en tabla `tickets` y `orders`
 - `ticket_root_id` y `version` en tabla `tickets`
 - `resale_status` y `owner_wallet` en tabla `resale`
 - `email` en tabla `users`
- Los índices ayudan a mejorar tiempos de respuesta en consultas de catálogo, historial de ventas y reventas P2P.
- El diagrama entidad-relación debe reflejar las relaciones principales:
 - `User → Order → Ticket`
 - `Event → Ticket`
 - `Ticket → Resale`

3.5.4. Uso de las Primary Key

Cada tabla tiene **primary key** única:

- `user_id` para `users`
- `event_id` para `events`
- `ticket_root_id + version` para `tickets`
- `order_id` para `orders`
- `resale_id` para `resale`

Permite consultas rápidas y evita ciclos o duplicidades, lo que garantiza integridad referencial con **foreign keys**:

- `tickets.event_id → events.event_id`
- `orders.user_id → users.user_id`
- `resale.ticket_id → tickets.ticket_root_id`

3.5.5. Frecuencia de acceso y conexiones

La base de datos debe soportar múltiples conexiones concurrentes, considerando:

- Clientes navegando y comprando boletos.
- Admin creando eventos y consultando contratos.
- Indexer sincronizando cada 5 segundos con blockchain.

Se recomienda **connection pool** con límite de 5 conexiones (`connection_limit=5`).

Consultas frecuentes y críticas:

- **SELECT** para catálogo y reventa P2P
- **INSERT** para órdenes y transacciones
- **UPDATE** para cambios de estado de boletos
- **DELETE** limitado solo a cancelaciones de carrito o reventa

El uso de **DAO** y **Prepared Statements** reduce la carga de procesamiento y protege contra inyección SQL.

4. Proceso de Ingeniería de Requerimientos

El proceso de ingeniería de requerimientos para el sistema *Stellar Tickets* se ha realizado siguiendo un enfoque iterativo y colaborativo, que asegura la claridad, trazabilidad y control sobre los cambios. Este proceso se complementa con los mecanismos de control definidos en el SPMP, que incluyen localización de requisitos, control de versiones y trazabilidad entre requerimientos, casos de uso y módulos del sistema.

4.1. Técnicas utilizadas

Análisis de stakeholders Se identificaron los actores principales: **CUSTOMER**, **ADMIN** y **STAFF**. Se registraron necesidades funcionales y no funcionales basadas en sus roles, permisos y flujo de interacción con el sistema.

Revisión de documentación existente Se analizaron referencias de proyectos similares y documentación de la red *Stellar*, contratos *Soroban* y estándares Web3. Se revisaron buenas prácticas de SPA, gestión de wallets y reventa de tickets digitales.

Modelado de casos de uso y diagramas Se construyeron diagramas de casos de uso que representan las funcionalidades principales del sistema y sus interacciones con los actores. Cada requerimiento funcional se vinculó con los casos de uso correspondientes para garantizar trazabilidad.

Descomposición por módulos y funcionalidades Los requerimientos se organizaron por módulos:

- Gestión de eventos y contratos

- Compra primaria de boletos
- Wallets e integración Web3
- Blockchain y NFTs
- Reventa P2P
- Validación QR y redención en puerta
- Administración de boletos

Documentación estructurada Se utilizó la Tabla 7 para todos los requerimientos funcionales y no funcionales. Se documentó: ID, tipo de requerimiento, casos de uso asociados, descripción, razón, autor, criterio de medición, prioridad, módulo asociado, versión y fecha. Se definieron criterios medibles de desempeño, seguridad, integridad, trazabilidad y disponibilidad para cada requisito.

Definición de requerimientos no funcionales y de desempeño Se establecieron límites claros para tiempos de respuesta, número de usuarios concurrentes, frecuencia de consultas, sincronización con blockchain y validación de QR. Se documentaron criterios de medición verificables para garantizar que el sistema cumpla con los estándares de calidad definidos.

4.2. Métodos de gestión de cambios

Control de versiones y trazabilidad Cada requerimiento tiene un ID único que permite localizarlo rápidamente. Los cambios se registran con la versión actualizada y la fecha, manteniendo historial de modificaciones.

Revisión y aprobación Todo cambio propuesto por el cliente o el equipo de desarrollo es revisado por los responsables del proyecto antes de ser aprobado. Se verifica que la modificación no afecte negativamente otros requerimientos ni la integridad del sistema.

Impacto en módulos y casos de uso Cada cambio se analiza respecto al módulo y caso de uso asociado. Esto asegura que se pueda actualizar la documentación de manera consistente y que los desarrolladores comprendan cómo afecta al sistema.

Integración con SPMP Los cambios se gestionan siguiendo las directrices del *Software Project Management Plan*, que establece los mecanismos de revisión, control y trazabilidad. Se asegura que los requerimientos actualizados se reflejen tanto en el SRS como en el SDD y en el modelo de dominio.

5. Proceso de Verificación

El proceso de verificación y validación del sistema *Stellar Tickets* se basa en asegurar que cada requerimiento funcional, no funcional y de desempeño cumpla con los criterios de calidad definidos en este SRS. Se aplican métodos tanto estáticos (revisión de documentación y consistencia) como dinámicos (pruebas funcionales, de carga y de integración con blockchain).

5.1. Verificación de requerimientos funcionales

Revisión de consistencia y trazabilidad

- Cada RF se cruza con su caso de uso correspondiente.
- Se revisa que todos los elementos documentados (usuario, módulo, entrada, proceso, salida) sean consistentes y completos.
- Se asegura que cada RF tenga un ID único y esté vinculado a su módulo.

Pruebas funcionales unitarias

- Cada funcionalidad (ej. creación de eventos, compra de boletos, reventa P2P, validación QR) se prueba con distintos test como se especifica en el documento de pruebas.
- Se verifica que las acciones del usuario produzcan los resultados esperados (por ejemplo, al comprar un boleto, se genera la orden y el ticket se asigna correctamente).

Pruebas de integración

- Se realizan pruebas que combinan varias funcionalidades, por ejemplo:
 - Compra primaria → Mis Entradas → Reventa P2P
 - Publicar boleto en reventa → Compra en reventa → Actualización de propiedad
- Se comprueba que la información entre módulos (frontend, backend, blockchain y base de datos) sea coherente.

Pruebas de aceptación del usuario (UAT)

- Se validan los flujos principales con usuarios simulados:
 - CUSTOMER: registro, compra, reventa, wallet Web3
 - ADMIN: creación de eventos, gestión de contratos
 - STAFF: validación de boletos mediante QR
- El objetivo es garantizar que el sistema cumple con los requerimientos funcionales definidos en la sección 3.

5.2. Verificación de requerimientos no funcionales y de desempeño

Pruebas de rendimiento

- Consultas al catálogo, carrito, checkout y sección *Mis Entradas* se prueban para asegurar tiempos de respuesta dentro de los límites definidos (por ejemplo, catálogo <3s, checkout <5s).
- Validación de QR en puerta se prueba bajo carga para garantizar respuesta <2s.

Pruebas de seguridad

- Validación de autenticación y roles (CUSTOMER, ADMIN, STAFF).
- Pruebas de acceso indebido a rutas protegidas, incluyendo endpoints Web3 y administración.
- Revisión de almacenamiento de información sensible: `password_hash`, `ORGANIZER_SECRET`, `JWT_SECRET` y `wallet_address`.

Pruebas de integridad y trazabilidad

- Verificación de que cada boleto comprado, revendido o invalidado mantiene historial completo en la base de datos y blockchain.
- Se ejecutan pruebas de reventa P2P para comprobar que *burn/remint* y actualización de propiedad funcionen correctamente.

Método	Descripción
Revisión de documentación	Verificar que todos los requerimientos estén completos, consistentes y trazables.
Pruebas unitarias	Evaluar funcionalidades individuales (registro, compra, publicación de boletos).
Pruebas de integración	Evaluar interacciones entre módulos y sincronización con blockchain.
Pruebas de carga	Simular múltiples usuarios concurrentes y medir desempeño de consultas y validaciones.
Pruebas de aceptación	Validar que las funcionalidades cumplen las necesidades de cada tipo de usuario.
Auditoría de blockchain	Confirmar que las transacciones reflejan correctamente la propiedad y estado de los boletos.

Cuadro 43: Métodos de verificación y validación

6. Referencias

Referencias

- [1] IEEE Std 830-1998, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Computer Society, 1998.
- [2] P. Kotler y K. Keller, “Dirección de Marketing,” 15^a edición, Pearson Educación, México, 2016.
- [3] Stellar Development Foundation, “Stellar Whitepaper,” [Online]. Available: <https://www.stellar.org/whitepaper>.
- [4] Soroban Documentation, Stellar Development Foundation, 2025. [Online]. Available: <https://soroban.stellar.org/docs>.
- [5] Freighter Wallet API Documentation, Stellar Development Foundation, 2024. [Online]. Available: <https://github.com/stellar/freighter-api>.
- [6] React Documentation, “React – A JavaScript library for building user interfaces,” Meta, 2023. [Online]. Available: <https://reactjs.org/docs/getting-started.html>.

- [7] TypeScript Team, “TypeScript: JavaScript With Syntax For Types,” Microsoft, 2023. [Online]. Available: <https://www.typescriptlang.org/docs/>.
- [8] Tailwind CSS Documentation, “Tailwind CSS: A Utility-First CSS Framework,” 2023. [Online]. Available: <https://tailwindcss.com/docs>.
- [9] Node.js Documentation, “Node.js v18.x,” OpenJS Foundation, 2023. [Online]. Available: <https://nodejs.org/en/docs/>.
- [10] Prisma Documentation, “Next-generation Node.js and TypeScript ORM,” 2023. [Online]. Available: <https://www.prisma.io/docs>.
- [11] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation,” 2023. [Online]. Available: <https://www.postgresql.org/docs/16/>.
- [12] TanStack Query, “TanStack Query Documentation,” 2023. [Online]. Available: <https://tanstack.com/query/v4/docs/overview>.
- [13] Framer Motion, “Framer Motion Documentation,” 2023. [Online]. Available: <https://www.framer.com/motion/>.